ATC Cookbook

Advanced Trigonometry Calculator Cookbook

Version: ATC 2.1.9 Language: English

This cookbook collects practical ATC workflows. It does not replace the full user guide; it gives copyable starting points for common work.

Examples marked as stable have deterministic output under the documented settings. Interactive modules are described as workflows because their exact screen text can depend on menu choices and persisted settings.

Quick Scientific Calculation

Problem: evaluate a short expression without creating variables.

Command(s):

```
2+3*4
sin(pi/2)
log(100)
```

Expected output:

```
14
1
2
```

When to use: quick one-line arithmetic, logarithmic and trigonometric checks.

Notes / limitations: output index numbers such as `#0` depend on previous commands in the current ATC session.

Trigonometry in Radians and Degrees

Problem: switch between radian and degree workflows.

Command(s):

```
mode
sin(pi/2)
sin(30)
```

Expected output: `sin(pi/2)` is normally used in radian mode; `sin(30)` is normally used in degree mode and returns `0.5`.

When to use: classroom trigonometry, engineering checks and solver experiments where angle units matter.

Notes / limitations: always confirm `mode` before comparing trigonometric results.

Quadratic Equation

Problem: solve a supported polynomial equation.

Command(s):

```
solve equation(x^2-5*x+6)
```

Expected output:

```
x1=3
x2=2
```

When to use: polynomial equations that ATC can normalize and route through the equation solver.

Notes / limitations: ATC is not a full CAS. Unsupported symbolic forms may fall back or be rejected.

Numerical Solver

Problem: find a root numerically when a direct polynomial equation command is not the right workflow.

Command(s):

```
solver(x+2)
solver(sin(x)-0.5)
```

Expected output: `solver(x+2)` returns `-2`. The trigonometric result depends on angle mode.

When to use: numerical root-finding for supported expressions.

Notes / limitations: check angle mode and avoid relying on undocumented parser behavior.

Polynomial Simplification

Problem: expand or simplify a supported polynomial expression.

Command(s):

```
simplify polynomial((x-2)*(x-3))
```

Expected output:

```
(1+0i)x^2+(_5+0i)x^1+(6+0i)
```

When to use: polynomial preparation before solving or documenting a calculation.

Notes / limitations: symbolic behavior exists where implemented and tested.

Roots to Polynomial

Problem: build a polynomial from known roots.

Command(s):

```
roots to polynomial(2\3)
```

Expected output: a second-degree polynomial with roots 2 and 3.

When to use: checking Vieta-style workflows or generating a polynomial for solver tests.

Notes / limitations: complex roots are supported in documented root forms.

Matrices

Problem: create and reuse a matrix.

Command(s):

```
create matrix(foo\2\2\3)
see variables
```

Expected output: ATC creates a matrix variable named `foo` and lists stored variables through `see variables`.

When to use: repeated matrix calculations or when examples need a named matrix.

Notes / limitations: matrix commands can be interactive. Prefer small matrices first while validating syntax.

Mean, Minimum and Maximum

Problem: calculate basic descriptive statistics from a list or matrix.

Command(s):

```
avg(2\4\6)
min(2\4\6)
max(2\4\6)
```

Expected output:

```
4
2
6
```

When to use: quick statistical summaries.

Notes / limitations: ATC 2.1.9 also validates `min`, `max` and `avg` with matrix variable names.

FFT / IFFT

Problem: run a simple DSP transform workflow.

Command(s):

```
fft(1\0\0\0)
ifft(1\1\1\1)
```

Expected output: ATC returns transform data in its documented numeric list format.

When to use: simple signal-processing checks and educational DSP examples.

Notes / limitations: verify scaling and ordering against the documented ATC format before using results in another tool.

TXT Workflow

Problem: process several ATC commands from a text file.

Command(s):

```
predefine txt
solve txt
eliminate strings
```

Expected output: ATC reads a text file, processes supported lines and writes an answer file.

When to use: batch calculations, repeatable examples and regression-style manual checks.

Notes / limitations: commands that open files or windows should be mocked or bypassed in automated tests.

Verbose Resolution for Debugging

Problem: inspect how ATC processes an expression.

Command(s):

```
verbose resolution(1)
sin(pi/6)
verbose resolution(0)
```

Expected output: ATC prints intermediate processing information while the setting is enabled.

When to use: parser and expression-debugging sessions.

Notes / limitations: verbose output is diagnostic text. Avoid using it as a stable mathematical output contract unless the regression suite explicitly covers that text.

Guided Modules

Problem: use ATC's menu-driven domain modules.

Command(s):

```
financial calculations
geometry calculations
statistics calculations
physics calculations
unit conversions
microeconomics calculations
triangles rectangles solver
arithmetic matrix solver
```

Expected output: each command opens its guided workflow.

When to use: when the calculation is easier to express through an ATC menu than as a one-line expression.

Notes / limitations: automated tests currently use safe smoke paths for these modules. Full per-option validation remains a manual or future automation area.